_combined_for_pdf

March 25, 2026

0.0.1 Sutton advection problem — general solver

This notebook implements and evaluates a 2-D boundary-layer advection model based on Sutton's problem, describing how temperature and water vapour redistribute downwind of an abrupt surface discontinuity (e.g. fallow → irrigated land).

What it does:

1. **Setup & physical constants** — imports from the `sutton` package, defines atmospheric constants (ρ, c_p, L_v, σ_{SB}), and sets boundary conditions .
2. **AdvectionParams dataclass** — parameter class with per-patch canopy geometry, energy partition, radiation closure, and implicit advection solvers for temperature and water vapour.
   - defines per-patch canopy geometry, flexible moisture specification (RH or Q), energy partition (H_f, LE_f, `avail_ratio`, `le_factor`), and methods to:
   - solve for incoming shortwave and albedo from energy balance (`solve_surface_radiation_inplace`),
   - derive surface boundary conditions from fluxes via log-law (`update_surface_BCs_from_reference`),
   - reconcile prescribed surface temperatures with sensible heat (`reconcile_heat_with_prescribed_surface_T`).
3. **Implicit advection solver** — marches temperature (`uniform_T`) and absolute humidity (`uniform_Q`) fields downwind using an implicit finite-difference scheme (Thomas algorithm), with mixing-length eddy diffusivity.
4. **Flux diagnostics** — converts kinematic fluxes to W m^{-2}, plots sensible and latent heat flux profiles along fetch, examines the Bowen ratio evolution, and fits exponential decay curves to the flux adjustment.
5. **2-D fields** — contour plots of Q and its vertical flux across the (x, z) domain.

'solve_surface_radiation_inplace(fix="alpha_c") calls a two-equation algebraic "radiation closure" that back-solves for an incoming shortwave value and an albedo so that the net radiation on each surface matches the available energy prescribed via R_n and G upwind and downwind of the transition.

1. Define target net radiation for each patch

- The routine first constructs the *target* net radiation for the fallow (upwind) and cultivated (downwind) surfaces

2. Assume a simple net-radiation model for each surface

For each surface type, the code uses:

$$R_{n,s} = SW\,(1-\alpha_s) \;+\; \varepsilon_a \sigma T_a^4 \;-\; \varepsilon_s \sigma T_s^4,$$

where - SW is incoming shortwave - α_s is the surface albedo - ε_a is an effective atmospheric emissivity, - ε_s is the surface emissivity, and - σ is the Stefan–Boltzmann constant.

Temperatures are converted to Kelvin internally:

$$T_K = T_{\mathrm{C}} + 273.15.$$

Thus the two balances are:

$$SW(1-\alpha_f) + \varepsilon_a \sigma T_a^4 - \varepsilon_f \sigma T_{sf}^4 = R_{n,f}$$

$$SW(1-\alpha_c) + \varepsilon_a \sigma T_a^4 - \varepsilon_c \sigma T_{sc}^4 = R_{n,c}$$

3) With `fix="alpha_c"`, α_c is treated as known (held at $alpha_c$), and the unknowns are SW α_f.

Sympy solves the resulting 2×2 system (linear in the unknowns).

Equivalently, the solution can be written explicitly as

$$SW = \frac{R_{n,c} - \varepsilon_a \sigma T_a^4 + \varepsilon_c \sigma T_{sc}^4}{1-\alpha_c}$$

$$\alpha_f = 1 - \frac{R_{n,f} - \varepsilon_a \sigma T_a^4 + \varepsilon_f \sigma T_{sf}^4}{SW}.$$

4. After solving, the routine updates the parameter object:

- `p.SW_in` $\leftarrow SW$,
- `p.alpha_f` $\leftarrow \alpha_f$ (clipped to $[0,1]$).

0.0.2 Including relative humidity

Convert vapor concentration Q (in $\mathrm{g\,m^{-3}}$) to vapor pressure e (Pa) using the ideal-gas law for water vapor:

$$e = \rho_v R_v T, \qquad \rho_v = \frac{Q}{1000},$$

where - ρ_v is water vapor density in $\mathrm{kg\,m^{-3}}$, - R_v is the specific gas constant for water vapor, and - T is absolute temperature (K).

Then compute relative humidity as

$$RH = 100\,\frac{e}{e_s(T)},$$

where $e_s(T)$ is the saturation vapor pressure at temperature T.

[ ]:
```
%%time
import numpy as np
import matplotlib.pyplot as plt
import sys
import math
import pandas as pd
```

```
CPU times: user 1.32 s, sys: 1.44 s, total: 2.76 s
Wall time: 930 ms
```

0.0.3 Parameter dataclass & solver functions

Extended parameter class with per-patch canopy geometry, energy partition, radiation closure, and implicit advection solvers for temperature and water vapour.

[ ]:
```
import sys
sys.path.insert(0, "/Users/octaviacrompton/Projects/sutton_advection/src")

# Project imports
from sutton import (
    AdvectionParams,
    Params,
    thomas,
    our_central_difference,
    no_central_difference,
    saturation_vapor_pressure,
    vapor_concentration_RH,
    SIGMA_SB, CP_AIR, RHO_AIR, LV, LV_G, RV,
)
```

0.0.4 Boundary conditions

[ ]:
```
# ---- Physical constants (aliased from sutton.constants for use in helper␣
↪cells below) ----
sigma_sb = SIGMA_SB   # W m^-2 K^-4  (Stefan-Boltzmann)
cp_air   = CP_AIR     # J kg^-1 K^-1
rho_air  = RHO_AIR    # kg m^-3
Lv       = LV         # J kg^-1
Lv_g     = LV_G       # J g^-1  (~30 °C)
Rv       = RV         # J kg^-1 K^-1  (water-vapour gas constant)

# They are NOT used by AdvectionParams - the dataclass has its own fields.
```

[ ]:
```
def print_radiation_budget(p: "AdvectionParams",
                           sigma: float = SIGMA_SB, digits: int = 2) -> dict:
    """
    Print (and return) the upwind (f) and downwind (c) radiation-budget terms
    consistent with p.solve_surface_radiation_inplace().
```

```
    Uses:
      Rn = SW*(1-alpha) + eps_a*sigma*Ta^4 - eps_s*sigma*Ts^4
      target Rn_f = RNmG_up + G
      target Rn_c = RNmG_down + G
    """
    def K(Tc): return Tc + 273.15

    SW   = float(p.SW_in)
    TaK  = K(float(p.T_a))
    TsfK = K(float(p.T_sf))
    TscK = K(float(p.T_sc))

    # Common atmospheric longwave down term in this simplified closure
    LW_down = float(p.epsilon_a) * sigma * TaK**4

    # --- Upwind / fallow (f) ---
    SWabs_f = SW * (1.0 - float(p.alpha_f))
    LWup_f  = float(p.epsilon_f) * sigma * TsfK**4
    Rn_f    = SWabs_f + LW_down - LWup_f
    RNmG_f  = Rn_f - float(p.G)
    Rn_f_target = float(p.RNmG_up) + float(p.G)

    # --- Downwind / cultivated (c) ---
    SWabs_c = SW * (1.0 - float(p.alpha_c))
    LWup_c  = float(p.epsilon_c) * sigma * TscK**4
    Rn_c    = SWabs_c + LW_down - LWup_c
    RNmG_c  = Rn_c - float(p.G)
    Rn_c_target = float(p.RNmG_down) + float(p.G)

    out = {
        "common": {
            "SW_in": SW,
            "Ta_K": TaK,
            "LW_down": LW_down,
            "G": float(p.G),
            "RNmG_up": float(p.RNmG_up),
            "RNmG_down": float(p.RNmG_down),
        },
        "upwind_f": {
            "alpha_f": float(p.alpha_f),
            "epsilon_f": float(p.epsilon_f),
            "Ts_K": TsfK,
            "SW_absorbed": SWabs_f,
            "LW_up": LWup_f,
            "Rn": Rn_f,
            "Rn_minus_G": RNmG_f,
```

```
            "Rn_target": Rn_f_target,
            "Rn_mismatch": Rn_f - Rn_f_target,
        },
        "downwind_c": {
            "alpha_c": float(p.alpha_c),
            "epsilon_c": float(p.epsilon_c),
            "Ts_K": TscK,
            "SW_absorbed": SWabs_c,
            "LW_up": LWup_c,
            "Rn": Rn_c,
            "Rn_minus_G": RNmG_c,
            "Rn_target": Rn_c_target,
            "Rn_mismatch": Rn_c - Rn_c_target,
        },
    }

    r = lambda x: float(np.round(x, digits))

    print("=== Radiation budget (closure-consistent) ===")
    print(f"SW_in = {r(SW)} W m^-2")
    print(f"Ta = {r(p.T_a)} °C  ({r(TaK)} K)")
    print(f"eps_a = {p.epsilon_a}  => LW_down = eps_a*sigma*Ta^4 = {r(LW_down)}␣
↪W m^-2")
    print(f"G = {r(p.G)} W m^-2")
    print(f"Targets: RNmG_up = {r(p.RNmG_up)} , RNmG_down = {r(p.RNmG_down)} ␣
↪(W m^-2)")
    print()

    print("--- Upwind (f) ---")
    print(f"alpha_f={p.alpha_f}, eps_f={p.epsilon_f}, Ts_f={r(p.T_sf)} °C␣
↪({r(TsfK)} K)")
    print(f"  SW_abs = SW*(1-alpha_f) = {r(SWabs_f)}")
    print(f"  LW_up  = eps_f*sigma*Ts^4 = {r(LWup_f)}")
    print(f"  Rn     = {r(Rn_f)}   |  Rn-G = {r(RNmG_f)}")
    print(f"  target Rn = RNmG_up + G = {r(Rn_f_target)}   | mismatch = {r(Rn_f␣
↪- Rn_f_target)}")
    print()

    print("--- Downwind (c) ---")
    print(f"alpha_c={p.alpha_c}, eps_c={p.epsilon_c}, Ts_c={r(p.T_sc)} °C␣
↪({r(TscK)} K)")
    print(f"  SW_abs = SW*(1-alpha_c) = {r(SWabs_c)}")
    print(f"  LW_up  = eps_c*sigma*Ts^4 = {r(LWup_c)}")
    print(f"  Rn     = {r(Rn_c)}   |  Rn-G = {r(RNmG_c)}")
    print(f"  target Rn = RNmG_down + G = {r(Rn_c_target)}   | mismatch =␣
↪{r(Rn_c - Rn_c_target)}")
```

```
    return out
```

[ ]:
```
def print_surface_moisture_check(p: "AdvectionParams", digits: int = 3) -> dict:
    """
    Print (and return) surface moisture consistency checks at the upwind (f) and
    downwind (c) surfaces using your conventions:

      T in °C
      Q in g m^-3 (absolute humidity / vapor concentration)
      RH in %

    It reports:
      - RH implied by (T, Q)
      - Q implied by (T, RH) (if RH is available)
      - mismatches vs stored RH/Q
      - saturation vapor concentration Q_sat(T) (g m^-3)
    """
    def K(Tc): return float(Tc) + 273.15

    # --- helpers (standalone so this works even if you refactor␣
↪AdvectionParams later) ---
    def RH_from_Q(Tc: float, Q_gm3: float) -> float:
        Tk = K(Tc)
        rho_v = float(Q_gm3) / 1000.0                # kg m^-3
        e = rho_v * Rv * Tk                          # Pa
        es = saturation_vapor_pressure(Tk)            # Pa
        return 100.0 * (e / es)

    def Q_from_RH(Tc: float, RH_pct: float) -> float:
        return float(vapor_concentration_RH(float(Tc), float(RH_pct)))  # g m^-3

    def r(x):
        if x is None:
            return None
        return float(np.round(float(x), digits))

    # pull stored values (RH_f may not exist yet)
    RHc_stored = getattr(p, "RH_c", None)
    RHf_stored = getattr(p, "RH_f", None)

    Tc, Tf = float(p.T_sc), float(p.T_sf)
    Qc, Qf = float(p.Q_c), float(p.Q_f)

    # implied from (T, Q)
    RHc_fromQ = RH_from_Q(Tc, Qc)
    RHf_fromQ = RH_from_Q(Tf, Qf)
```

```
    # implied from (T, RH) if RH exists
    Qc_fromRH = Q_from_RH(Tc, RHc_stored) if RHc_stored is not None else None
    Qf_fromRH = Q_from_RH(Tf, RHf_stored) if RHf_stored is not None else None

    # saturation vapor concentration at each surface
    Qsat_c = Q_from_RH(Tc, 100.0)
    Qsat_f = Q_from_RH(Tf, 100.0)

    out = {
        "downwind_c": {
            "T_C": Tc, "T_K": K(Tc),
            "Q_gm3": Qc,
            "RH_stored_pct": RHc_stored,
            "RH_from_Q_pct": RHc_fromQ,
            "RH_mismatch_pct": (RHc_fromQ - float(RHc_stored)) if RHc_stored is␣
↪not None else None,
            "Q_from_RH_gm3": Qc_fromRH,
            "Q_mismatch_gm3": (Qc_fromRH - Qc) if Qc_fromRH is not None else␣
↪None,
            "Q_sat_gm3": Qsat_c,
        },
        "upwind_f": {
            "T_C": Tf, "T_K": K(Tf),
            "Q_gm3": Qf,
            "RH_stored_pct": RHf_stored,
            "RH_from_Q_pct": RHf_fromQ,
            "RH_mismatch_pct": (RHf_fromQ - float(RHf_stored)) if RHf_stored is␣
↪not None else None,
            "Q_from_RH_gm3": Qf_fromRH,
            "Q_mismatch_gm3": (Qf_fromRH - Qf) if Qf_fromRH is not None else␣
↪None,
            "Q_sat_gm3": Qsat_f,
        }
    }

    print("=== Surface moisture check (T, Q, RH) ===")
    print("Conventions: T in °C, Q in g m^-3, RH in %")
    print()

    # --- downwind (c) ---
    c = out["downwind_c"]
    print("--- Downwind surface (c) ---")
    print(f"T_sc = {r(c['T_C'])} °C  ({r(c['T_K'])} K)")
    print(f"Q_c  = {r(c['Q_gm3'])} g m^-3   | Q_sat(T_sc) = {r(c['Q_sat_gm3'])}␣
↪g m^-3")
    print(f"RH implied by (T_sc, Q_c): {r(c['RH_from_Q_pct'])} %")
```

```
    if c["RH_stored_pct"] is not None:
        print(f"RH stored: {r(c['RH_stored_pct'])} %   | mismatch =␣
↪{r(c['RH_mismatch_pct'])} %")
        print(f"Q implied by (T_sc, RH_stored): {r(c['Q_from_RH_gm3'])} g m^-3"
              f"   | mismatch vs Q_c = {r(c['Q_mismatch_gm3'])} g m^-3")
    else:
        print("RH stored: None (only reporting RH implied by Q)")
    print()

    # --- upwind (f) ---
    f = out["upwind_f"]
    print("--- Upwind surface (f) ---")
    print(f"T_sf = {r(f['T_C'])} °C  ({r(f['T_K'])} K)")
    print(f"Q_f  = {r(f['Q_gm3'])} g m^-3   | Q_sat(T_sf) = {r(f['Q_sat_gm3'])}␣
↪g m^-3")
    print(f"RH implied by (T_sf, Q_f): {r(f['RH_from_Q_pct'])} %")
    if f["RH_stored_pct"] is not None:
        print(f"RH stored: {r(f['RH_stored_pct'])} %   | mismatch =␣
↪{r(f['RH_mismatch_pct'])} %")
        print(f"Q implied by (T_sf, RH_stored): {r(f['Q_from_RH_gm3'])} g m^-3"
              f"   | mismatch vs Q_f = {r(f['Q_mismatch_gm3'])} g m^-3")
    else:
        print("RH stored: None (only reporting RH implied by Q)")

    return out
```

[ ]:
```
level = 1  # initial solver level; redefined as _level in downstream cells
ustar_f = 0.75
shear_ratio = 1
ustar_c = ustar_f*np.sqrt(shear_ratio)

p = AdvectionParams(ustar_f = ustar_f,
                    ustar_c = ustar_c,
                    lm_option = 'kz',
                    lm_zshift = 0.0005,
                    zom_c = 0.0005,
                    avail_ratio =  1.8,
                    Hmax = 2,
                    le_factor = 1.35,
                    dz = 0.005,
                    dx = 0.2,
                    Q_a = 6.5,
                    Q_f = 6.5,
                    tie_Qa_to_Qf=True,
          )

print (p.alpha_f, p.T_sc, p.Q_c, p.RH_c)
```

```
out = p.solve_surface_radiation_inplace(fix="alpha_c")

print (p.alpha_f, p.T_sf, "Q_f", p.Q_f,  p.Q_a, p.Q_c, p.RH_c)
out = p.update_surface_BCs_from_reference(update_T=True, update_Q=True)
print (p.alpha_f, p.T_sf, "Q_f", p.Q_f,  p.Q_a, p.Q_c, p.RH_c)
p.sync_surface_moisture_inplace("RH_from_Q")
print (p.H_f, p.T_sf, p.T_sc, p.Q_f,  p.Q_a, p.Q_c)

p.reconcile_heat_with_prescribed_surface_T(mode="update_ustar", ref="z_ref")
print (p.ustar_f, p.T_sf, p.T_sc, p.Q_f, p.Q_a, p.Q_c)

out = p.update_surface_BCs_from_reference(update_T=True, update_Q=True)
print (p.ustar_f, p.T_sf, p.T_sc, p.Q_f, p.Q_a, p.Q_c)
```

```
0.22 28.5 17.168590668180276 59.99944070242963
0.4860160718532113 40.0 Q_f 6.5 6.5 17.168590668180276 59.99944070242963
0.4860160718532113 38.152835306815604 Q_f 6.5 6.5 16.33231938001962
70.2093668628321
355.64 38.152835306815604 24.86371375670617 6.5 6.5 16.33231938001962
0.7499999999999998 38.152835306815604 24.86371375670617 6.5 6.5
16.33231938001962
0.7499999999999998 38.152835306815604 24.86371375670617 6.5 6.5
16.33231938001962
```

[ ]:
```
# Turbulent fluxes at the downwind wall (x = 0) from log-law gradients.
# These verify that the surface BC fluxes are consistent with Rn - G.
H_wall_Wm2  =  p.ustar_c * p.k * (p.T_sc - p.T_a) / np.log(p.Hmax / p.z[0]) *␣
 ↪rho_air * cp_air

Fq_wall_gm2s = p.ustar_c * p.k * (p.Q_c - p.Q_a) / np.log(p.Hmax / p.z[0])
LE_wall_Wm2  = Fq_wall_gm2s * Lv_g
```

[ ]:
```
# Available energy check: LE + H at wall (should  Rn - G)
LE_wall_Wm2 + H_wall_Wm2
```

[ ]: 640.1520000000002

[ ]:
```
# Moisture flux at crop surface from log law (g m^-2 s^-1)
wQ = float(p.k) * float(p.ustar_c) * (float(p.Q_a) - float(p.Q_c)) / np.log(p.
 ↪Hmax/p.z[0])
```

[ ]:
```
wQ
```

[ ]: -0.35564000000000007

[ ]:
```
# TODO: migrate integrate_H2O_step, integrate_T_step, uniform_Q, uniform_T,
# get_upwind_fluxQ, get_upwind_fluxT to src/sutton/integrators.py.
# Kept inline for now for rapid iteration.
```

```python
def integrate_H2O_step(p, Q_up, A, B, C):
    """
    One x-step implicit solve for absolute humidity Q (g m^-3).
    BCs: surface Q = p.Q_c, aloft Q = p.Q_a.
    A, B, C : arrays length nz
    """
    m   = int(p.nz)
    dx  = p.dx
    dz  = p.dz
    z   = p.z

    AA1 = -A * B
    AA2 = -C * B
    AA3 = 1 / dx
    AA4 = Q_up / dx

    upd = (AA1 / (dz ** 2) + AA2 / (2 * dz))
    dia = (-2 * AA1 / (dz ** 2) + AA3)
    lod = (AA1 / (dz ** 2) - AA2 / (2 * dz))

    co = np.zeros(m)
    co[:] = AA4

    lod[0] = lod[m - 1] = 0
    dia[0] = dia[m - 1] = 1
    upd[0] = upd[m - 1] = 0

    co[0] = p.Q_c
    co[m - 1] = p.Q_a

    Q1 = thomas(lod, dia, upd, co)
    dQdz = no_central_difference(Q1, z)

    Fq = - A / z * dQdz
    return Q1, Fq


def integrate_T_step(p, T_up, A, B, C):
    """
    One x-step implicit solve for temperature T (°C).
    BCs: surface T = p.T_sc, aloft T = p.T_a.
    """
    m   = int(p.nz)
    dx  = p.dx
    dz  = p.dz
    z   = p.z
```

```
    AA1 = -A * B
    AA2 = -C * B
    AA3 = 1 / dx
    AA4 = T_up / dx

    upd = (AA1 / (dz ** 2) + AA2 / (2 * dz))
    dia = (-2 * AA1 / (dz ** 2) + AA3)
    lod = (AA1 / (dz ** 2) - AA2 / (2 * dz))

    co = np.zeros(m)
    co[:] = AA4

    lod[0] = lod[m - 1] = 0
    dia[0] = dia[m - 1] = 1
    upd[0] = upd[m - 1] = 0

    co[0] = p.T_sc
    co[-1] = p.T_a

    T1 = thomas(lod, dia, upd, co)
    dTdz = no_central_difference(T1, z)

    FT = - (A / z) * dTdz
    return T1, FT


def uniform_Q(p):
    """
    Implicit solution for water-vapor concentration Q (g m^-3) and its
    mass flux (g m^-2 s^-1), marched along x.
    """
    nx, nz = p.nx, p.nz
    dx, dz = p.dx, p.dz
    zmax   = p.Hmax
    k      = p.k
    ustar_f, ustar_c = p.ustar_f, p.ustar_c
    Q_f, Q_c, Q_a = p.Q_f, p.Q_c, p.Q_a
    z = p.z

    FQ_f = ustar_f * k * (Q_f - Q_a) / np.log(zmax / z[0])
    Qup  = Q_f - (FQ_f / (k * ustar_f)) * np.log(z / z[0])

    U = (ustar_f / k) * np.log(z / (z[0] + 1e-9))

    lm = p.lm
    A  = lm * ustar_c
```

```
    B  = 1.0 / U
    C  = our_central_difference(A, dz)

    Q1 = Qup.copy()
    Q_uniform     = np.zeros((nx, nz))
    FluxQ_uniform = np.zeros((nx, nz))
    Q_uniform[0, :] = Q1

    for i in range(nx):
        Q2, Fq = integrate_H2O_step(p, Q1, A, B, C)
        Q_uniform[i, :]     = Q2
        FluxQ_uniform[i, :] = Fq
        Q1 = Q2

    return Q_uniform, FluxQ_uniform


def uniform_T(p):
    """
    Implicit solution for temperature T (°C) and its kinematic heat flux
    (K m s^-1), marched along x.
    """
    nx, nz = p.nx, p.nz
    dx, dz = p.dx, p.dz
    zmax   = p.Hmax
    k      = p.k
    ustar_f, ustar_c = p.ustar_f, p.ustar_c
    T_sf, T_sc, T_a = p.T_sf, p.T_sc, p.T_a
    z = p.z

    wT_up = -(T_a - T_sf) * k * ustar_f / np.log(zmax / z[0])
    Tup   = T_sf - (wT_up / (k * ustar_f)) * np.log(z / z[0])

    U = (ustar_f / k) * np.log(z / (p.zom_c + 1e-9))

    lm = p.lm
    A  = lm * ustar_c
    B  = 1.0 / U
    C  = our_central_difference(A, dz)

    T1 = Tup.copy()
    T_uniform     = np.zeros((nx, nz))
    FluxT_uniform = np.zeros((nx, nz))
    T_uniform[0, :] = T1

    for i in range(nx):
        T2, FT = integrate_T_step(p, T1, A, B, C)
```

```
        T_uniform[i, :]     = T2
        FluxT_uniform[i, :] = FT
        T1 = T2

    return T_uniform, FluxT_uniform


def get_upwind_fluxQ(p):
    """
    Build the upwind Q profile (g m^-3) and its diffusive mass flux
    (g m^-2 s^-1) using neutral two-point logs.
    """
    z     = p.z
    dz    = p.dz
    zmax  = p.Hmax
    k     = p.k
    ustar = p.ustar_f
    Q_f, Q_a = p.Q_f, p.Q_a

    FQ_f = ustar * k * (Q_f - Q_a) / np.log(zmax / z[0])
    Q_up = Q_f - (FQ_f / (k * ustar)) * np.log(z / z[0])

    lm = p.lm
    A  = lm * ustar

    dQdz = no_central_difference(Q_up, z)
    FluxQ_upwind = -A / z * dQdz

    return Q_up, FluxQ_upwind


def get_upwind_fluxT(p):
    """
    Build the upwind temperature profile T_up (°C) and its kinematic
    heat-flux profile (K m s^-1) using neutral two-point logs.
    """
    z     = p.z
    dz    = p.dz
    zmax  = p.Hmax
    k     = p.k
    ustar = p.ustar_f
    T_sf, T_a = p.T_sf, p.T_a

    wT_up = -(T_a - T_sf) * k * ustar / np.log(zmax / z[0])
    T_up  = T_sf - (wT_up / (k * ustar)) * np.log(z / z[0])

    lm = p.lm
```

```
    A  = lm * ustar

    dTdz = no_central_difference(T_up, z)
    FluxT_upwind = -A / z * dTdz

    return T_up, FluxT_upwind
```

[ ]:
```
# The model is initialized by prescribing RH and updating Q
# out = p.update_surface_BCs_from_zom()
# print (p.Q_c, p.LE_c, p.T_sc,  p.T_sf)
out = p.update_surface_BCs_from_reference()
print (p.Q_c, p.LE_c, p.T_sc,  p.T_sf)
print (p.alpha_f, p.T_sf, p.Q_f, p.RH_f)
T_uniform, FluxT_uniform = uniform_T(p)
Q_uniform, FluxQ_uniform = uniform_Q(p)

T_up, FluxT_upwind = get_upwind_fluxT(p)
Q_up, FluxQ_upwind = get_upwind_fluxQ(p)

H_uniform_Wm2 = rho_air * cp_air * FluxT_uniform     # → W m^-2
LE_uniform_Wm2 = Lv_g * FluxQ_uniform  # → W m^-2
FluxTotal = H_uniform_Wm2 + LE_uniform_Wm2

H_upwind_Wm2 = rho_air * cp_air * FluxT_upwind     # → W m^-2
LE_upwind_Wm2 = Lv_g * FluxQ_upwind  # → W m^-2
FluxTotal_upwind = H_upwind_Wm2 + LE_upwind_Wm2

x = p.x
FluxTotal_upwind = LE_upwind_Wm2[level]*np.ones_like(x) + \
            H_upwind_Wm2[level]*np.ones_like(x)
```

```
16.33231938001962 864.2052000000001 24.86371375670617 38.152835306815604
0.4860160718532113 38.152835306815604 6.5 13.429901844005435
```

0.0.5 Run advection solver

Solve for T and Q fields, compute sensible and latent heat fluxes.

[ ]:
```
print (p.alpha_f, p.T_sf, p.Q_f, p.RH_f)
```

```
0.4860160718532113 38.152835306815604 6.5 13.429901844005435
```

[ ]:
```
def print_surface_energy_closure_from_profiles(p: "AdvectionParams", *,
    rho_air: float = RHO_AIR,   # kg m^-3
    cp_air: float = CP_AIR,     # J kg^-1 K^-1
    Lv_g: float = LV_G,         # J g^-1  (so Q-flux in g m^-2 s^-1 -> W m^-2)
    sigma: float = SIGMA_SB,
    digits: int = 2,) -> dict:
```

```
    """
    Check energy closure using fluxes computed *from the
    prescribed neutral log profiles*:

      w'T'  = -k u_* (T_a - T_s) / ln(zmax / z_surf)
      w'Q'  =  k u_* (Q_s - Q_a) / ln(zmax / z_surf)

    Convert to energy fluxes:
      H  = rho cp w'T'
      LE = Lv_g * w'Q'   (Lv_g in J g^-1; w'Q' in g m^-2 s^-1)

    Then compare against radiation (same simplified form as␣
↪solve_surface_radiation_inplace):
      Rn = SW(1-alpha) + eps_a*sigma*T_a^4 - eps_s*sigma*T_s^4
      check:  Rn - G ?= H + LE

    Notes:
    - Uses z_surf = p.z[0] and zmax = p.Hmax to match your uniform_T/uniform_Q␣
↪setup.
    - Positive H, LE are upward (surface -> air).
    """
    def K(Tc): return float(Tc) + 273.15
    r = lambda x: float(np.round(float(x), digits))

    # geometry / log factor
    z_surf = float(p.z[0])
    zmax   = float(p.Hmax)
    logfac = float(np.log(zmax / z_surf))

    # radiation inputs
    SW  = float(p.SW_in)
    TaK = K(p.T_a)
    LW_down = float(p.epsilon_a) * sigma * TaK**4

    # --- helper to compute fluxes from the prescribed two-point log profile ---
    def fluxes_from_log_profile(Ts, Qa, Qs, ustar):
        # kinematic heat flux (K m s^-1)
        wT = - (float(p.T_a) - float(Ts)) * float(p.k) * float(ustar) / logfac
        # moisture mass flux (g m^-2 s^-1)
        wQ =   float(p.k) * float(ustar) * (float(Qs) - float(Qa)) / logfac
        # energy fluxes (W m^-2)
        H  = rho_air * cp_air * wT

        LE = Lv_g * wQ
        return wT, wQ, H, LE
```

```
    # --- Upwind / fallow (f) ---
    wT_f, wQ_f, H_f_prof, LE_f_prof = fluxes_from_log_profile(
        p.T_sf, p.Q_a, p.Q_f, p.ustar_f)

    TsfK   = K(p.T_sf)
    Rn_f   = SW * (1.0 - float(p.alpha_f)) + LW_down - float(p.epsilon_f) *␣
↪sigma * TsfK**4
    avail_f = Rn_f - float(p.G)
    sum_f   = H_f_prof + LE_f_prof
    resid_f = avail_f - sum_f

    # --- Downwind / cultivated (c) ---
    Q_c = float(p.Q_c)  # may be cached_property or overridden in __dict__
    wT_c, wQ_c, H_c_prof, LE_c_prof = fluxes_from_log_profile(
        p.T_sc, p.Q_a, Q_c, p.ustar_c)

    TscK   = K(p.T_sc)
    Rn_c   = SW * (1.0 - float(p.alpha_c)) + LW_down - float(p.epsilon_c) *␣
↪sigma * TscK**4
    avail_c = Rn_c - float(p.G)
    sum_c   = H_c_prof + LE_c_prof
    resid_c = avail_c - sum_c

    # optional: compare to your target "prescribed" available energies, if␣
↪present
    RNmG_up   = float(getattr(p, "RNmG_up", np.nan))
    RNmG_down = float(getattr(p, "RNmG_down", np.nan))

    out = {
        "common": {
            "z_surf": z_surf, "zmax": zmax, "ln(zmax/z_surf)": logfac,
            "SW_in": SW, "LW_down": LW_down, "G": float(p.G),
        },
        "upwind_f": {
            "ustar": float(p.ustar_f), "T_s_C": float(p.T_sf), "Q_s_gm3":␣
↪float(p.Q_f),
            "alpha": float(p.alpha_f), "epsilon_s": float(p.epsilon_f),
            "wT_Kms": float(wT_f), "wQ_gm2s": float(wQ_f),
            "H_Wm2": float(H_f_prof), "LE_Wm2": float(LE_f_prof),
            "Rn_Wm2": float(Rn_f), "Rn_minus_G": float(avail_f),
            "H_plus_LE": float(sum_f), "residual": float(resid_f),
            "target_RNmG": RNmG_up,
        },
        "downwind_c": {
            "ustar": float(p.ustar_c), "T_s_C": float(p.T_sc), "Q_s_gm3":␣
↪float(Q_c),
            "alpha": float(p.alpha_c), "epsilon_s": float(p.epsilon_c),
```

```
            "wT_Kms": float(wT_c), "wQ_gm2s": float(wQ_c),
            "H_Wm2": float(H_c_prof), "LE_Wm2": float(LE_c_prof),
            "Rn_Wm2": float(Rn_c), "Rn_minus_G": float(avail_c),
            "H_plus_LE": float(sum_c), "residual": float(resid_c),
            "target_RNmG": RNmG_down,
        },
    }

    def pct(resid, denom):
        denom = float(denom)
        if denom == 0.0:
            return np.nan
        return 100.0 * float(resid) / denom

    print("=== Surface energy closure from prescribed log profiles ===")
    print("Check:  Rn - G  ?=  H + LE   (W m^-2)")
    print(f"z_surf = {(z_surf)} m, zmax = {r(zmax)} m, ln(zmax/z_surf) =␣
↪{r(logfac)}")
    print(f"SW_in = {r(SW)} | LW_down = eps_a*sigma*Ta^4 = {r(LW_down)} | G =␣
↪{r(p.G)}")
    print()

    print("--- Upwind (f) ---")
    print(f"u*_f = {r(p.ustar_f)} m s^-1 | T_sf={r(p.T_sf)} °C | Q_f={r(p.Q_f)}␣
↪g m^-3 | Q_a={r(p.Q_a)} g m^-3")
    print(f"Fluxes from log profile: w'T'={r(wT_f)} K m s^-1  ->␣
↪H={r(H_f_prof)} W m^-2")
    print(f"                         w'Q'={r(wQ_f)} g m^-2 s^-1 ->␣
↪LE={r(LE_f_prof)} W m^-2")
    print(f"Radiation: Rn={r(Rn_f)} | Rn-G={r(avail_f)}")
    print(f"Closure:   H+LE={r(sum_f)} | residual={(r(resid_f))} ␣
↪({r(pct(resid_f, avail_f))}% of Rn-G)")
    if np.isfinite(RNmG_up):
        print(f"Target RNmG_up={r(RNmG_up)} | mismatch ( (H+LE) - target ) =␣
↪{r(sum_f - RNmG_up)}")
    print()

    print("--- Downwind (c) ---")
    print(f"u*_c = {r(p.ustar_c)} m s^-1 | T_sc={r(p.T_sc)} °C | Q_c={r(Q_c)} g␣
↪m^-3 | Q_a={r(p.Q_a)} g m^-3")
    print(f"Fluxes from log profile: w'T'={r(wT_c)} K m s^-1  ->␣
↪H={r(H_c_prof)} W m^-2")
    print(f"                         w'Q'={r(wQ_c)} g m^-2 s^-1 ->␣
↪LE={r(LE_c_prof)} W m^-2")
    print(f"Radiation: Rn={r(Rn_c)} | Rn-G={r(avail_c)}")
```

```
    print(f"Closure:   H+LE={r(sum_c)} | residual={(r(resid_c))} ␣
↪({r(pct(resid_c, avail_c))}% of Rn-G)")
    if np.isfinite(RNmG_down):
        print(f"Target RNmG_down={r(RNmG_down)} | mismatch ( (H+LE) - target )␣
↪= {r(sum_c - RNmG_down)}")

    return out

eb = print_surface_energy_closure_from_profiles(p)
```

```
=== Surface energy closure from prescribed log profiles ===
Check:  Rn - G  ?=  H + LE   (W m^-2)
z_surf = 0.0005 m, zmax = 2.0 m, ln(zmax/z_surf) = 8.29
SW_in = 901.34 | LW_down = eps_a*sigma*Ta^4 = 383.12 | G = 0.0

--- Upwind (f) ---
u*_f = 0.75 m s^-1 | T_sf=38.15 °C | Q_f=6.5 g m^-3 | Q_a=6.5 g m^-3
Fluxes from log profile: w'T'=0.29 K m s^-1  -> H=355.64 W m^-2
                      w'Q'=0.0 g m^-2 s^-1 -> LE=0.0 W m^-2
Radiation: Rn=367.12 | Rn-G=367.12
Closure:   H+LE=355.64 | residual=11.48  (3.13% of Rn-G)
Target RNmG_up=355.64 | mismatch ( (H+LE) - target ) = -0.0

--- Downwind (c) ---
u*_c = 0.75 m s^-1 | T_sc=24.86 °C | Q_c=16.33 g m^-3 | Q_a=6.5 g m^-3
Fluxes from log profile: w'T'=-0.19 K m s^-1  -> H=-224.05 W m^-2
                      w'Q'=0.36 g m^-2 s^-1 -> LE=864.21 W m^-2
Radiation: Rn=661.27 | Rn-G=661.27
Closure:   H+LE=640.15 | residual=21.12  (3.19% of Rn-G)
Target RNmG_down=640.15 | mismatch ( (H+LE) - target ) = 0.0
```

[ ]:
```
print_radiation_budget(p);
moist = print_surface_moisture_check(p)
```

```
=== Radiation budget (closure-consistent) ===
SW_in = 901.34 W m^-2
Ta = 30.0 °C  (303.15 K)
eps_a = 0.8  => LW_down = eps_a*sigma*Ta^4 = 383.12 W m^-2
G = 0.0 W m^-2
Targets: RNmG_up = 355.64 , RNmG_down = 640.15  (W m^-2)

--- Upwind (f) ---
alpha_f=0.4860160718532113, eps_f=0.9, Ts_f=38.15 °C (311.3 K)
  SW_abs = SW*(1-alpha_f) = 463.28
  LW_up  = eps_f*sigma*Ts^4 = 479.28
  Rn     = 367.12   |  Rn-G = 367.12
  target Rn = RNmG_up + G = 355.64   | mismatch = 11.48
```

```
--- Downwind (c) ---
alpha_c=0.22, eps_c=0.95, Ts_c=24.86 °C (298.01 K)
  SW_abs = SW*(1-alpha_c) = 703.05
  LW_up  = eps_c*sigma*Ts^4 = 424.89
  Rn     = 661.27   |  Rn-G = 661.27
  target Rn = RNmG_down + G = 640.15   | mismatch = 21.12
=== Surface moisture check (T, Q, RH) ===
Conventions: T in °C, Q in g m^-3, RH in %

--- Downwind surface (c) ---
T_sc = 24.864 °C  (298.014 K)
Q_c  = 16.332 g m^-3   | Q_sat(T_sc) = 23.262 g m^-3
RH implied by (T_sc, Q_c): 70.209 %
RH stored: 70.209 %   | mismatch = 0.0 %
Q implied by (T_sc, RH_stored): 16.332 g m^-3   | mismatch vs Q_c = -0.0 g m^-3

--- Upwind surface (f) ---
T_sf = 38.153 °C  (311.303 K)
Q_f  = 6.5 g m^-3   | Q_sat(T_sf) = 48.399 g m^-3
RH implied by (T_sf, Q_f): 13.43 %
RH stored: 13.43 %   | mismatch = 0.0 %
Q implied by (T_sf, RH_stored): 6.5 g m^-3   | mismatch vs Q_f = -0.0 g m^-3
```

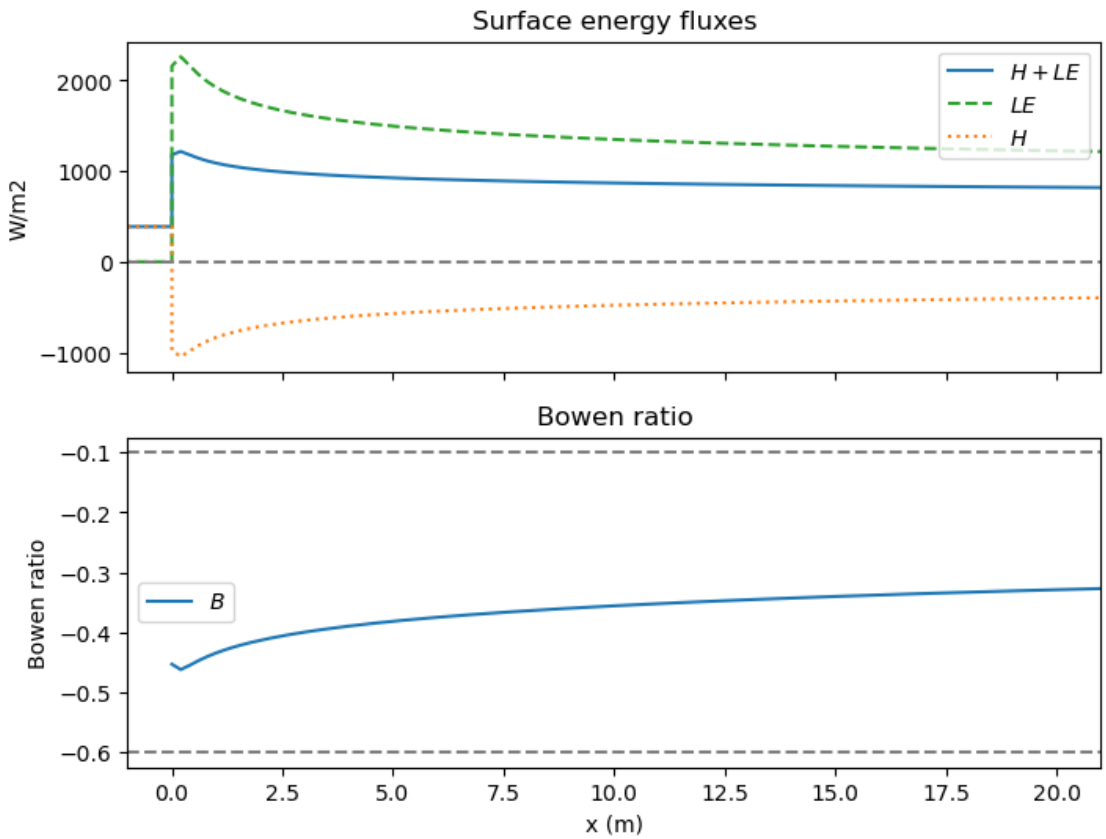
[ ]:
```
# Turbulent fluxes at the upwind "wall" (x = 0) from log-law gradients
# across the full domain height.  These are the fluxes the flow carries
# into the downwind (crop) domain at the leading edge.
#   H_wall  = rho * cp * u*_c * k * (T_s - T_a) / ln(z_top / z0)
#   LE_wall = Lv * u*_c * k * (Q_s - Q_a) / ln(z_top / z0)
H_wall_Wm2  =  p.ustar_c * p.k * (p.T_sc - p.T_a) / np.log(p.Hmax / p.z[0]) *␣
 ↪rho_air *cp_air
Fq_wall_gm2s = p.ustar_c * p.k * (p.Q_c - p.Q_a) / np.log(p.Hmax / p.z[0])
LE_wall_Wm2  = Fq_wall_gm2s * Lv_g

# Total energy flux at the wall (should approximate Rn - G if closure holds)
H_wall_Wm2 + LE_wall_Wm2
```

[ ]: 640.1520000000002

[ ]:
```
# --- Two-panel figure: surface energy fluxes (top) and Bowen ratio (bottom) ---
_level = level  # use the solver-level value from the main configuration cell
fig, axes = plt.subplots( 2, 1, figsize = (8, 6 ), sharex = True)

# --- Top panel: H, LE, and H+LE along the fetch ---
ax = axes[0]

# Combine upwind (negative x) and downwind (positive x) into a single transect
xx = np.concatenate([-np.flip(x),x])
```

```
FluxTotal_ = np.concatenate([FluxTotal_upwind, FluxTotal[:, _level]])

LE_Wm2 = np.concatenate([ LE_upwind_Wm2[_level]*np.ones_like(x),␣
 ↪LE_uniform_Wm2[:, _level]])
H_Wm2 = np.concatenate([ H_upwind_Wm2[_level]*np.ones_like(x), H_uniform_Wm2[:,␣
 ↪_level]])

# Model curves
ax.plot(xx, FluxTotal_,  c =  'C0', ls = '-', label = '$H + LE$')
ax.plot(xx, LE_Wm2,  ls = '--', c =  'C2',  label = "$LE$")
ax.plot(xx, H_Wm2,  ls = ':', c =  'C1',  label = "$H$")
ax.set_xlim(-1, 21)
ax.legend()

# Rao (1974) Figure 4 reference points (circles) at x = 0, 2, 20 m


ax.set_ylabel("W/m2")
ax.set_title("Surface energy fluxes")
ax.axhline(0, ls = '--', c = 'grey')

# --- Bottom panel: Bowen ratio B = H / LE along the downwind fetch ---
ax = axes[1]
ax.plot(x, (H_uniform_Wm2[:, _level])/LE_uniform_Wm2[:, _level], label = "$B$")
ax.set_title("Bowen ratio")
plt.xlabel("x (m)")
plt.ylabel("Bowen ratio")
# Asymptotic reference lines for B
ax.axhline(-0.1, ls = '--', c = 'grey')
ax.axhline(-0.6, ls = '--', c = 'grey')
plt.legend()
```

[ ]: <matplotlib.legend.Legend at 0x14798d990>



0.0.6 Flux diagnostics

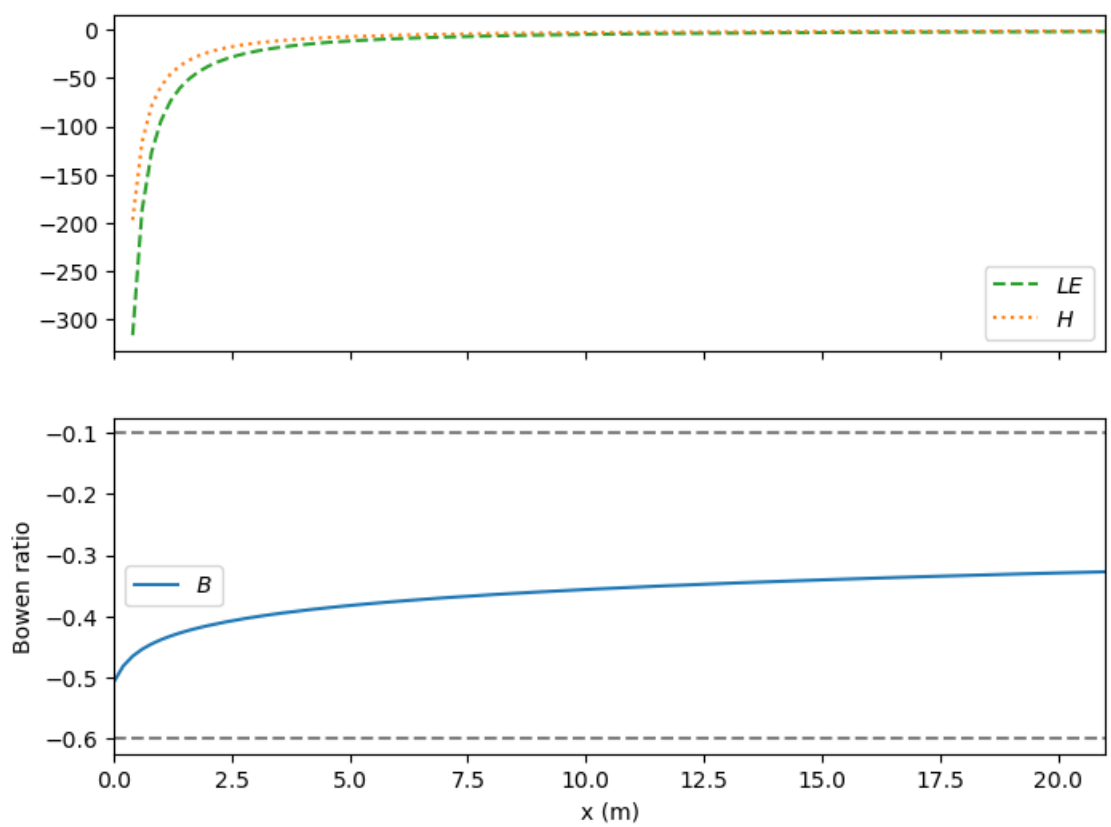
Sensible and latent heat flux profiles along fetch, Bowen ratio, and energy balance checks.

0.0.7 Surface-level T, Q, and Bowen ratio along fetch

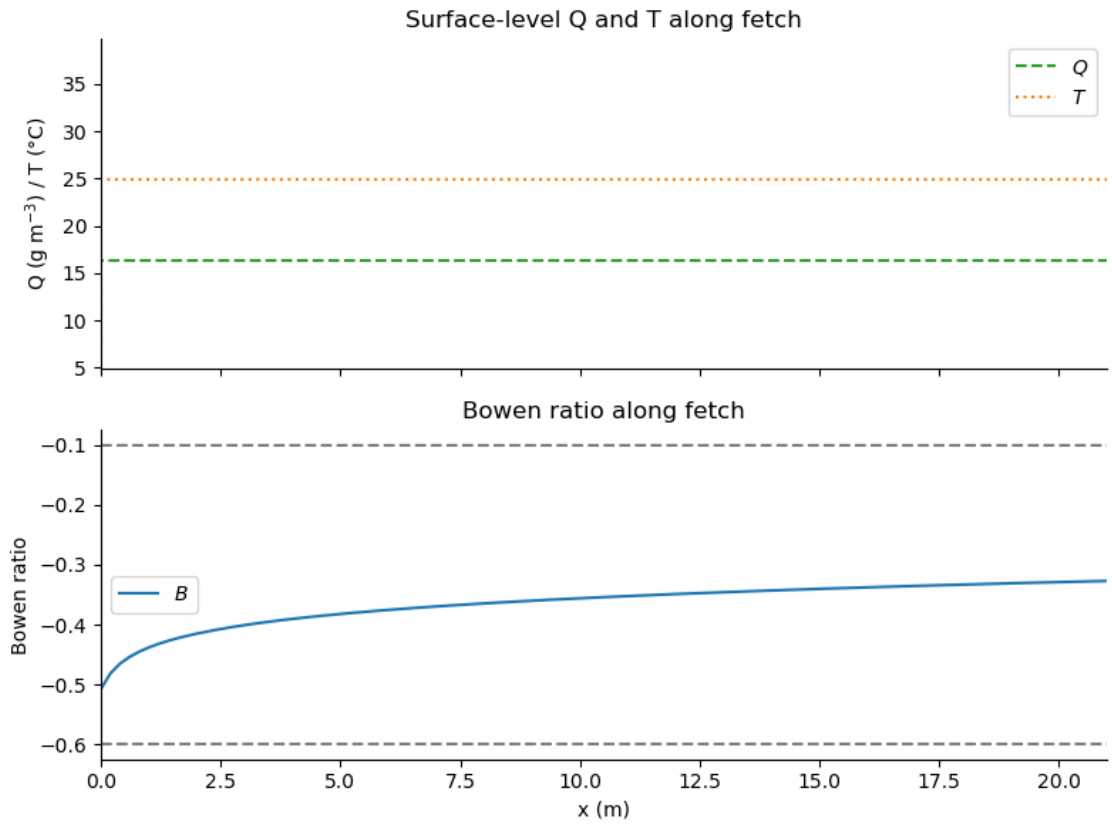
[ ]:
```
_level = 0  # surface fluxes
fig, axes = plt.subplots( 2, 1, figsize = (8, 6 ), sharex = True)
ax = axes[0]

xx = np.concatenate([-np.flip(x),x])
FluxTotal_ = np.concatenate([FluxTotal_upwind, FluxTotal[:, _level]])

LE_Wm2 = np.concatenate([ LE_upwind_Wm2[_level]*np.ones_like(x),␣
↪LE_uniform_Wm2[:, _level]])
H_Wm2 = np.concatenate([ H_upwind_Wm2[_level]*np.ones_like(x), H_uniform_Wm2[:,␣
↪_level]])

ax.plot(p.x[2:], np.diff(LE_uniform_Wm2[1:,_level]),  ls = '--', c =  'C2', ␣
↪label = "$LE$")
```

```
ax.plot(p.x[2:], - np.diff(H_uniform_Wm2[1:, _level]),  ls = ':', c =  'C1', ␣
 ↪label = "$H$")
ax.set_xlim(0, 21)

# ax.set_yscale('log')
ax.legend()

ax = axes[1]
x = p.x

ax.plot(x, (H_uniform_Wm2[:, _level])/LE_uniform_Wm2[:, _level], label = "$B$")
plt.xlabel("x (m)")
plt.ylabel("Bowen ratio")
ax.axhline(-0.1, ls = '--', c = 'grey')
ax.axhline(-0.6, ls = '--', c = 'grey')

plt.legend()
# Sensible heat flux needs to move to 0 faster than LE moves to LE_c?
```

[ ]: <matplotlib.legend.Legend at 0x147c19410>



[ ]:
```
fig, axes = plt.subplots(2, 1, figsize=(8, 6), sharex=True)

xx = np.concatenate([-np.flip(x), x])

_level = 0  # surface-level T, Q, and Bowen ratio
Q = np.concatenate([Q_up[_level] * np.ones_like(x), Q_uniform[:, _level]])
T = np.concatenate([T_up[_level] * np.ones_like(x), T_uniform[:, _level]])

# --- top panel: Q and T at surface level ---
ax = axes[0]
ax.plot(xx[1:], Q[1:], ls='--', c='C2', label="$Q$")
ax.plot(xx[1:], T[1:], ls=':', c='C1', label="$T$")
ax.set_xlim(0, 21)
ax.set_ylabel("Q (g m$^{-3}$) / T (°C)")
ax.set_title("Surface-level Q and T along fetch")
ax.legend()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

# --- bottom panel: Bowen ratio ---
ax = axes[1]
ax.plot(p.x, H_uniform_Wm2[:, _level] / LE_uniform_Wm2[:, _level], label="$B$")
ax.axhline(-0.1, ls='--', c='grey')
ax.axhline(-0.6, ls='--', c='grey')
ax.set_xlabel("x (m)")
ax.set_ylabel("Bowen ratio")
ax.set_title("Bowen ratio along fetch")
ax.legend()
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

plt.tight_layout()
```



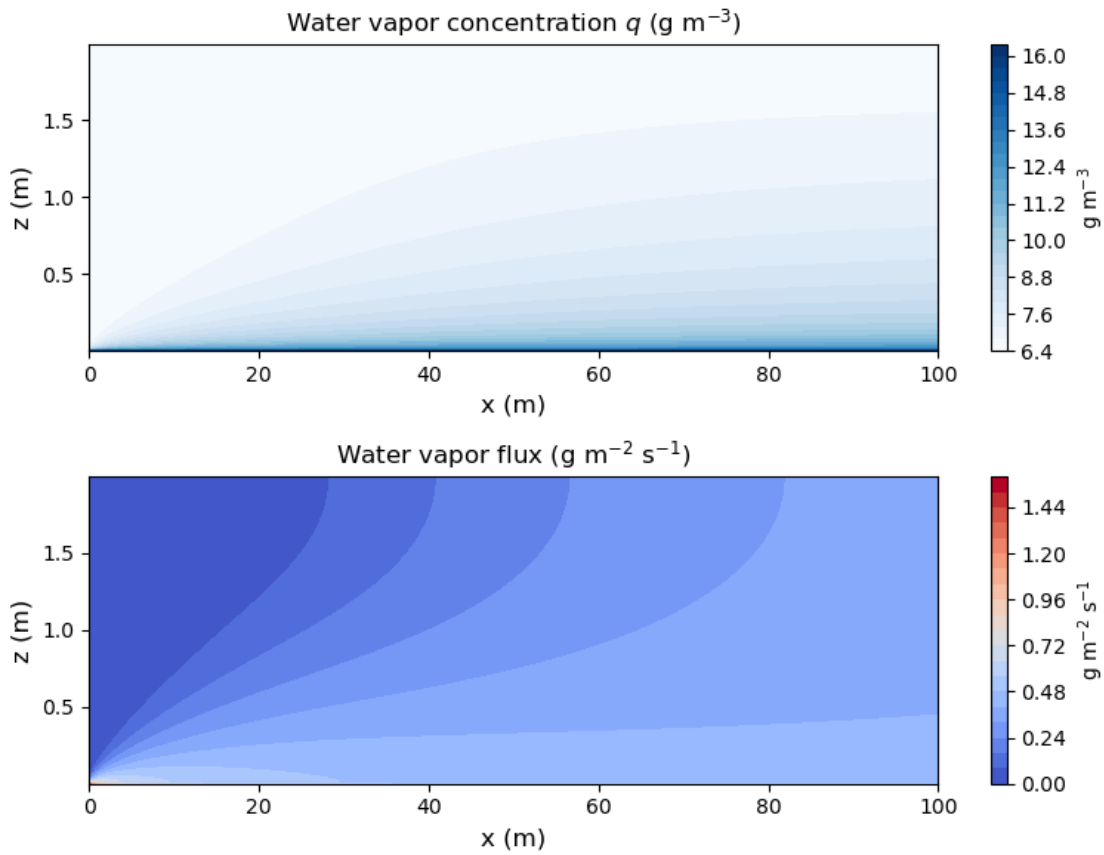
[ ]:
```
# Upwind checks

H_wall_Wm2  =  p.ustar_f * p.k * (p.T_sf - p.T_a) / np.log(p.Hmax / p.z[0]) *␣
 ↪rho_air *cp_air

print("H_f (AdvectionParams):       ", p.H_f)
print("H_wall:              {0:.2f}".format( H_wall_Wm2) )


# Downwind checks
# Lv_g = 2430.0  # J g^-1
Fq_wall_gm2s = p.ustar_c * p.k * (p.Q_c - p.Q_a) / np.log(p.Hmax / p.z[0])
LE_wall_Wm2  = Fq_wall_gm2s * Lv_g

print("\nLE_c (AdvectionParams):      ", p.LE_c)
print("LE_wall:             {0:.2f} ".format(LE_wall_Wm2))
```

```
H_f (AdvectionParams):        355.64
H_wall:              355.64

LE_c (AdvectionParams):       864.2052000000001
```

```
LE_wall:              864.21
```

$$
\begin{aligned}
\mathrm{SW}\,(1-\alpha_f) + \varepsilon_a\,\sigma_{SB}\,T_a^4 - \varepsilon_f\,\sigma_{SB}\,T_{sf}^4 &= R_{n,f}, \\
\mathrm{SW}\,(1-\alpha_c) + \varepsilon_a\,\sigma_{SB}\,T_a^4 - \varepsilon_c\,\sigma_{SB}\,T_{sc}^4 &= R_{n,c}, \\
T_{sf} - T_a &= \frac{1}{k\,u_{*f}}\,\frac{H_f}{\rho\,c_p}\,\ln\left(\frac{z_h}{z_\ell}\right), \\
T_{sc} - T_a &= \frac{1}{k\,u_{*c}}\,\frac{H_c}{\rho\,c_p}\,\ln\left(\frac{z_h}{z_\ell}\right), \\
Q_c - Q_a &= \frac{1}{k\,u_{*c}}\,\frac{\mathrm{LE}_c}{L_v}\,\ln\left(\frac{z_h}{z_\ell}\right), \\
R_{n,c} - G &= \mathrm{LE}_c + H_c.
\end{aligned}
$$

[ ]:
```python
z = p.z
x = p.x

# Plot water vapor concentration and vertical flux
plt.figure(figsize = (8, 6))

plt.subplot(2, 1, 1)
nz = Q_uniform.shape[1]
plt.contourf(x, z[:nz], (Q_uniform.T), 25, cmap='Blues',  vmax = p.Q_c, vmin =p.
 ↪Q_a)
plt.colorbar(label='g m$^{-3}$')
plt.xlabel('x (m)', fontsize=12, fontweight='normal')
plt.ylabel('z (m)', fontsize=12, fontweight='normal')
plt.title('Water vapor concentration $q$ (g m$^{-3}$)', fontsize=12)

plt.subplot(2, 1, 2)
plt.contourf(x, z[:nz], FluxQ_uniform.T.round(4),  20, cmap='coolwarm', vmin =␣
 ↪0, vmax = FluxQ_uniform.max())
plt.colorbar(label='g m$^{-2}$ s$^{-1}$')
plt.xlabel('x (m)', fontsize=12, fontweight='normal')
plt.ylabel('z (m)', fontsize=12, fontweight='normal')
plt.title('Water vapor flux (g m$^{-2}$ s$^{-1}$)', fontsize=12)
plt.tight_layout()
```



0.0.8 2-D fields

Contour plots of Q and vertical moisture flux across the (x, z) domain.

[ ]:
```
_level = 1  # first interior grid point for rough estimate
# rough estimate at level height z:
z_eff = z[_level] - p.h_c*2/3
lm    = p.k * z_eff
K     = lm * ustar_c
U = (p.ustar_f / p.k) * np.log(z / p.zom_c)
Lrec  = U[_level] * z_eff / (p.k * ustar_c)
```

[ ]:
```
Q_uniform.shape, p.z.shape
```

[ ]: ((501, 400), (400,))

0.1 Appendix — `sutton` package source code

The cells below contain the full source of every module in `src/sutton/` that this notebook imports from.

0.1.1 `src/sutton/constants.py`

Physical constants

[ ]:
```
"""
Physical constants used throughout the sutton package.

Import from here rather than defining ad-hoc module-level globals.
"""

# Stefan-Boltzmann constant (W m^-2 K^-4)
SIGMA_SB: float = 5.670374419e-8

# Dry-air properties (standard conditions, ~20 °C, 1 atm)
CP_AIR: float = 1005.0     # J kg^-1 K^-1  (specific heat at constant pressure)
RHO_AIR: float = 1.20      # kg m^-3

# Latent heat of vaporisation (~30 °C)
LV: float = 2.43e6         # J kg^-1
LV_G: float = 2430.0       # J g^-1   (= LV / 1000)

# Water-vapour gas constant
RV: float = 461.5           # J kg^-1 K^-1
```

0.1.2 `src/sutton/physics.py`

Physics functions (saturation vapour pressure, vapour concentration)

[ ]:
```
import math
import numpy as np


def saturation_vapor_pressure(T):
    """
    Saturation vapor pressure over liquid water (Pa) via Clausius-Clapeyron
    with constant latent heat.

    Parameters
    ----------
    T : float or array-like
        Temperature in Kelvin.

    Returns
    -------
```

```
    float or np.ndarray
        Saturation vapor pressure in Pa (same shape as T).
    """
    T = np.asarray(T, dtype=float)
    e0 = 611.2      # Pa at T0
    t0 = 273.15     # K
    rv = 461.5      # J kg^-1 K^-1
    lv = 2.501e6    # J kg^-1  (latent heat of vaporization, constant)
    es = e0 * np.exp((lv / rv) * (1.0 / t0 - 1.0 / T))
    return es if es.shape else float(es)



def vapor_concentration_RH_Tetens(T_c, RH=100.0):
    T_c = np.asarray(T_c, dtype=float)
    RH  = np.clip(np.asarray(RH, dtype=float), 0.0, 100.0)

    # Tetens (T in °C): e_s in kPa → ×1000 to Pa
    e_s = 0.6108 * np.exp((17.27 * T_c) / (T_c + 237.3)) * 1000.0  # Pa
    e   = (RH / 100.0) * e_s

    M_w = 18.015  # g mol^-1
    R   = 8.314   # J mol^-1 K^-1
    C = (e * M_w) / (R * (T_c + 273.15))  # g m^-3

    return C if C.shape else float(C)



def vapor_concentration_RH(T_c, RH=100.0):
    """
    Absolute humidity (g m^-3) at temperature T_c (°C) and relative humidity RH␣
 ↪(%).
    Uses saturation_vapor_pressure(T_K) and ideal gas law.
    """
    T_c = np.asarray(T_c, dtype=float)

    # saturation vapor pressure (Pa) using CC function (expects Kelvin)
    e_s = saturation_vapor_pressure(T_c + 273.15)  # Pa

    # partial pressure of water vapor (Pa)
    e = (RH / 100.0) * e_s

    # absolute humidity C = rho_v (g m^-3) using ideal gas:  rho = (e * M_w) /␣
 ↪(R * T)
    M_w = 18.015    # g mol^-1
    R   = 8.314     # J mol^-1 K^-1
```

```
    C = (e * M_w) / (R * (T_c + 273.15))   # g m^-3

    return C if C.shape else float(C)



def vapor_concentration(es: float, T: float) -> float:
    M = 18.015
    R = 8.314
    rho = (es * M) / (R * T)
    return float(rho)


def e_sat(T: float) -> float:
    e_s = 0.6108 * math.exp((17.27 * T) / (T + 237.3)) * 1000
    return float(e_s)
```

0.1.3 src/sutton/numerics.py

Numerics (Thomas algorithm, central difference)

[ ]:
```
import numpy as np


def thomas(aa, bb, cc, dd):
    """
    Tridiagonal matrix solver (Thomas algorithm).
    aa: lower diag, bb: main diag, cc: upper diag, dd: RHS
    Returns solution vector q.
    """
    n = len(bb)
    bet = np.zeros(n)
    gam = np.zeros(n)
    q = np.zeros(n)

    bet[0] = bb[0]
    gam[0] = dd[0] / bb[0]

    for i in range(1, n):
        bet[i] = bb[i] - (aa[i] * cc[i - 1] / bet[i - 1])
        gam[i] = (dd[i] - aa[i] * gam[i - 1]) / bet[i]

    q[n - 1] = gam[n - 1]
    for i in range(n - 2, -1, -1):
        q[i] = gam[i] - (cc[i] * q[i + 1] / bet[i])
    return q
```

```python
def thomas_alt(aa, bb, cc, dd):
    """
    Alternate Thomas algorithm implementation.
    """
    n = len(bb)
    gam = np.zeros(n)
    q = np.zeros(n)

    bet = bb[0]
    q[0] = dd[0] / bet

    for i in range(1, n):
        gam[i] = cc[i - 1] / bet
        bet = bb[i] - aa[i] * gam[i]
        if bet == 0:
            break
        q[i] = (dd[i] - aa[i] * q[i - 1]) / bet

    for i in range(n - 2, -1, -1):
        q[i] = q[i] - gam[i + 1] * q[i + 1]
    return q


def our_central_difference(s, dz):
    """
    Central differences; forward/backward at edges.
    """
    m = len(s)
    dsdz = np.zeros(m)
    dsdz[1:m - 1] = (s[2:m] - s[0:m - 2]) / (2 * dz)
    dsdz[0] = (s[1] - s[0]) / dz
    dsdz[m - 1] = (s[m - 1] - s[m - 2]) / dz
    return dsdz


def no_central_difference(s, z):
    """
    Central differences; forward/backward at edges.
    """
    m = len(z)
    dz = np.ones(m)
    ds = np.ones(m)
    dz[:-1] = np.log(z[1:] / z[:-1])
    ds[:-1] = (s[1:] - s[:-1])
    ds_dz = ds/dz
    ds_dz[-1] = ds_dz[-2]
```

```
    return ds_dz
```

0.1.4 src/sutton/stability.py

Stability corrections

[ ]:
```
import numpy as np


def stability(xi):
    """Brutsaert stability function (stable/unstable)."""
    xi = np.asarray(xi)
    phi = np.zeros_like(xi)
    inds = np.where(xi <= 1 / 16)
    phi[inds] = (1 - 16 * xi[inds]) ** -0.5
    inds = np.where(xi > 1 / 16)
    phi[inds] = 1 + 5.2 * xi[inds]
    return phi
```

0.1.5 src/sutton/utils.py

Utilities

[ ]:
```
import numpy as np


def padit(x, nz, upwind, array):
    xx = np.concatenate([np.arange(-40, 0, 2), x])
    up = upwind.reshape(1, nz)
    up = np.concatenate([up] * 10)
    array = np.concatenate([up, up, array])
    return xx, array
```

0.1.6 src/sutton/integrators.py

Integrators (integrate_T_implicit, integrate_H2O_implicit)

[ ]:
```
import numpy as np
from .numerics import thomas, our_central_difference, no_central_difference



def integrate_H2O_implicit(n, m, dx, dz, A, B, C, Qup, Qs, Qa, z):
    AA1 = -A * B
    AA2 = -C * B
    AA3 = 1 / dx
    AA4 = Qup / dx
```

```
    upd = (AA1 / (dz ** 2) + AA2 / (2 * dz))
    dia = (-2 * AA1 / (dz ** 2) + AA3)
    lod = (AA1 / (dz ** 2) - AA2 / (2 * dz))

    co = np.zeros(m)
    co[:] = AA4

    lod[0] = lod[m - 1] = 0
    dia[0] = dia[m - 1] = 1
    upd[0] = upd[m - 1] = 0

    co[0] = Qs
    co[m - 1] = Qa

    Q1 = thomas(lod, dia, upd, co)
    dQdz = no_central_difference(Q1, z)

    Fq = - A/z * dQdz

    return Q1, Fq

def integrate_T_implicit(n, m, dx, dz, A, B, C, Tup, Ts, Ta, z):

    AA1 = -A * B
    AA2 = -C * B
    AA3 = 1 / dx
    AA4 = Tup / dx

    upd = (AA1 / (dz ** 2) + AA2 / (2 * dz))
    dia = (-2 * AA1 / (dz ** 2) + AA3)
    lod = (AA1 / (dz ** 2) - AA2 / (2 * dz))

    co = np.zeros(m)
    co[:] = AA4

    lod[0] = lod[m - 1] = 0
    dia[0] = dia[m - 1] = 1
    upd[0] = upd[m - 1] = 0

    co[0] = Ts
    co[-1] = Ta

    T1 = thomas(lod, dia, upd, co)
    dTdz = no_central_difference(T1, z)

    FT = - A/z * dTdz
```

```
    return T1, FT
```

0.1.7 src/sutton/config.py

AdvectionParams dataclass (main parameter class)

[ ]:
```
"""
Parameter dataclasses for Sutton's advection problem.

``AdvectionParams``
    Extended parameter class with per-patch canopy geometry, energy
    partition, radiation closure, and boundary-condition methods for
    the implicit advection solver.

``Params``
    Legacy parameter class (kept for backward compatibility with older
    notebooks).  New code should use ``AdvectionParams``.
"""

from __future__ import annotations

from dataclasses import dataclass, asdict, fields, InitVar
from functools import cached_property
from typing import Optional

import numpy as np

from .constants import SIGMA_SB, CP_AIR, RHO_AIR, LV_G, RV
from .physics import saturation_vapor_pressure, vapor_concentration_RH


# ---------------------------------------------------------------------------
# AdvectionParams  (migrated from *Advection - general.ipynb*, March 2026)
# ---------------------------------------------------------------------------

@dataclass
class AdvectionParams:
    """Extended parameter dataclass for the two-patch advection model.

    Stores all grid, surface, meteorology, and energy-partition parameters.
    Surface moisture is stored canonically as ``Q_c`` / ``Q_f`` (g m^-3);
    ``RH_c`` / ``RH_f`` are read-only ``@property`` accessors derived from
    (T, Q) and are never stored.

    Typical usage
    -------------
```

```
    >>> p = AdvectionParams(ustar_f=0.8, ustar_c=0.8, le_factor=1.4,
    ...                     avail_ratio=1.4, Hmax=2, dz=0.005, dx=0.2)
    >>> p.solve_surface_radiation_inplace(fix="alpha_c")
    >>> p.update_surface_BCs_from_reference(update_T=True, update_Q=True)
    >>> p.sync_surface_moisture_inplace("RH_from_Q")
    >>> p.reconcile_heat_with_prescribed_surface_T(mode="update_ustar",
    ...                                            ref="z_ref")
    >>> p.update_surface_BCs_from_reference(update_T=True, update_Q=True)
    """

    # --- core physics / numerics ---
    k: float = 0.4
    zom_f: float = 0.0005            # m  (fallow / upwind momentum roughness)
    zom_c: float = 0.0005            # m  (crop / downwind momentum roughness)

    # canopy & displacement inputs (per patch)
    # (If None -> derive from z0m via alpha_m = 0.1, disp_frac)
    h_f_opt: Optional[float] = None
    h_c_opt: Optional[float] = None
    d_f_opt: Optional[float] = None
    d_c_opt: Optional[float] = None

    alpha_m: float = 0.10            # z0m ~ alpha_m * h
    disp_frac: float = 0.67          # d   ~ disp_frac * h

    G: float = 0.0                   # ground heat flux  (W m^-2)

    # domain / grid
    Lx: float = 100.0                # m
    Hmax: float = 10.0               # m
    dz: float = 0.1                  # m
    dx: float = 1.0                  # m
    xmin: float = 0.0                # m

    # friction velocities (optional; None => compute via resolve_ustars)
    ustar_f: Optional[float] = 0.15
    ustar_c: Optional[float] = 0.15

    # block pattern
    fallow_fraction: float = 0.5
    fallow_length: float = 1000.0    # m

    # temperatures (deg C)
    T_sc: float = 28.5
    T_sf: float = 40.0
    T_a:  float = 30.0
```

```
    # ---- surface moisture (choose ONE per surface at init) ----
    # Provide either RH_* (%) or Q_* (g m^-3).  If neither is given,
    # defaults (RH_c=60 %, RH_f=10 %) are used.
    # RH_* are InitVar: accepted at __init__ but NOT stored as fields.
    # After construction, p.RH_c / p.RH_f are computed @property.
    RH_c: InitVar[Optional[float]] = None
    RH_f: InitVar[Optional[float]] = None
    Q_c:  Optional[float] = None     # g m^-3  (canonical stored field)
    Q_f:  Optional[float] = None     # g m^-3  (canonical stored field)

    Q_a: float = 6.0
    tie_Qa_to_Qf: bool = False

    # radiation inputs
    SW_in: float = 400.0
    epsilon_f: float = 0.90
    epsilon_c: float = 0.95
    epsilon_a: float = 0.80
    alpha_c: float = 0.22
    alpha_f: float = 0.22

    lm_option: str | None = "kz"
    lm_zshift: float = 0.0005        # m, additive shift; tuned to match Rao␣
↪(1974)

    # reference wind for u* inference (if computed from log law)
    U_ref: float = 3.76              # m s^-1 at z_ref_wind
    z_ref_wind: float = 4.0          # m

    # upstream / base flux: Rao base case uses (Rn-G)_up = H_f
    H_f: float = 355.64              # ~ 356 W m^-2
    LE_f: float = 0.0                # upstream LE (W m^-2)

    # downstream available-energy ratio & partition
    avail_ratio: float = 1.0         # (Rn-G)_down / (Rn-G)_up
    le_factor: float = 1.0           # LE_c = le_factor * (Rn-G)_down

    # ------------------------------------------------------------------
    #  helpers & grids
    # ------------------------------------------------------------------

    @staticmethod
    def _K(Tc: float) -> float:
        """deg C -> K."""
        return Tc + 273.15

    # derived canopy height & displacement (per patch) ----------------
```

```python
    @cached_property
    def h_f(self) -> float:
        if self.h_f_opt is not None:
            return float(self.h_f_opt)
        if self.d_f_opt is not None:
            return float(self.d_f_opt) / self.disp_frac
        return float(self.zom_f) / self.alpha_m

    @cached_property
    def h_c(self) -> float:
        if self.h_c_opt is not None:
            return float(self.h_c_opt)
        if self.d_c_opt is not None:
            return float(self.d_c_opt) / self.disp_frac
        return float(self.zom_c) / self.alpha_m

    @cached_property
    def d_f(self) -> float:
        if self.d_f_opt is not None:
            return float(self.d_f_opt)
        if self.h_f_opt is not None:
            return self.disp_frac * float(self.h_f_opt)
        return self.disp_frac * (float(self.zom_f) / self.alpha_m)

    @cached_property
    def d_c(self) -> float:
        if self.d_c_opt is not None:
            return float(self.d_c_opt)
        if self.h_c_opt is not None:
            return self.disp_frac * float(self.h_c_opt)
        return self.disp_frac * (float(self.zom_c) / self.alpha_m)

    # grid ------------------------------------------------------------

    @cached_property
    def zmin(self) -> float:
        """Conservative global lower bound: z > max(z0m_f, z0m_c)."""
        return max(self.zom_f, self.zom_c)

    @cached_property
    def z(self) -> np.ndarray:
        return np.arange(self.zmin, self.Hmax, self.dz)

    @cached_property
    def x(self) -> np.ndarray:
        return np.arange(self.xmin, self.Lx + self.dx, self.dx)
```

```python
    @property
    def nz(self) -> int:
        return len(self.z)

    @property
    def nx(self) -> int:
        return len(self.x)

    # friction velocity ------------------------------------------------

    def resolve_ustars(self) -> "AdvectionParams":
        """Compute u* from log law if not already set."""
        if self.ustar_f is None:
            assert self.z_ref_wind > self.zom_f > 0
            self.ustar_f = self.k * self.U_ref / np.log(self.z_ref_wind / self.
↪zom_f)
        if self.ustar_c is None:
            assert self.z_ref_wind > self.zom_c > 0
            self.ustar_c = self.k * self.U_ref / np.log(self.z_ref_wind / self.
↪zom_c)
        return self

    # block sizes ------------------------------------------------------

    @cached_property
    def fallow_size(self) -> int:
        return int(self.fallow_length / self.dx)

    @cached_property
    def field_size(self) -> int:
        return (
            int(self.fallow_size
                * (1 - self.fallow_fraction) / self.fallow_fraction)
            if self.fallow_fraction > 0 else 0
        )

    # mixing length ----------------------------------------------------

    @cached_property
    def lm(self) -> np.ndarray:
        d = 2 / 3 * self.h_c
        z = self.z
        k = self.k
        h = self.h_c

        if self.lm_option == "kz":
```

```
        lm = k * (z + self.lm_zshift)
    else:
        a = 1
        lm = k * z.copy()
        lm[z < a * (h - d)] = k * a * h / 3
    return lm

# saturation vapour pressure (cached) ------------------------------

@cached_property
def es_c(self) -> float:
    return saturation_vapor_pressure(self._K(self.T_sc))

@cached_property
def es_f(self) -> float:
    return saturation_vapor_pressure(self._K(self.T_sf))

# RH: always derived from canonical (T, Q) state -------------------

@property
def RH_c(self) -> float:
    """Relative humidity at cultivated surface (%), from (T_sc, Q_c)."""
    return self.RH_from_Q(self.T_sc, self.Q_c)

@property
def RH_f(self) -> float:
    """Relative humidity at fallow surface (%), from (T_sf, Q_f)."""
    return self.RH_from_Q(self.T_sf, self.Q_f)

# energy partition -------------------------------------------------

@property
def RNmG_up(self) -> float:
    """Upstream available energy (W m^-2)."""
    return self.H_f + self.LE_f

@property
def RNmG_down(self) -> float:
    """Downstream available energy = avail_ratio * RNmG_up (W m^-2)."""
    return self.RNmG_up * self.avail_ratio

@property
def LE_c(self) -> float:
    """Downstream latent heat flux (W m^-2).

    ``le_factor > 1`` is valid and gives negative ``H_c``
    (evaporative cooling, T_sc < T_a).
```

```
        """
        return self.le_factor * self.RNmG_up * self.avail_ratio

    @property
    def H_c(self) -> float:
        """Downstream sensible heat flux (W m^-2)."""
        return (1.0 - self.le_factor) * self.RNmG_up * self.avail_ratio

    # ------------------------------------------------------------------
    #  to_dict (legacy call-sites)
    # ------------------------------------------------------------------

    def to_dict(self) -> dict:
        d = asdict(self)
        d.update({
            "x": self.x,
            "z": self.z,
            "nx": self.nx,
            "nz": self.nz,
            "zmin": self.zmin,
            "zmax": self.Hmax,
            "xmax": self.Lx,
            "ustar_f": self.ustar_f,
            "ustar_c": self.ustar_c,
            "fallow_size": self.fallow_size,
            "field_size": self.field_size,
            "Q_c": self.Q_c,
            "Q_f": self.Q_f,
            "Q_a": self.Q_a,
            "RH_c": self.RH_c,
            "RH_f": self.RH_f,
            "es_c": self.es_c,
            "es_f": self.es_f,
            "LE_c": self.LE_c,
            "H_c": self.H_c,
            "RNmG_up": self.RNmG_up,
            "RNmG_down": self.RNmG_down,
            "h_f": self.h_f,
            "h_c": self.h_c,
            "d_f": self.d_f,
            "d_c": self.d_c,
        })
        return d

    # ------------------------------------------------------------------
    #  Radiation closure
    # ------------------------------------------------------------------
```

```python
    def solve_surface_radiation_inplace(self, *, fix: str = "alpha_f") -> dict:
        """Solve the two-surface net-radiation balances, updating in place.

        The system is::

            Rn_f = SW*(1 - alpha_f) + eps_a * sigma * T_a^4 - eps_f * sigma *␣
↪T_sf^4
            Rn_c = SW*(1 - alpha_c) + eps_a * sigma * T_a^4 - eps_c * sigma *␣
↪T_sc^4

        with  Rn_f = RNmG_up + G  and  Rn_c = RNmG_down + G.

        One albedo is kept fixed (``fix``); the other and SW are solved.

        Parameters
        ----------
        fix : {"alpha_f", "alpha_c"}
            Which albedo to treat as prescribed.

        Returns
        -------
        dict with SW, alpha_f, alpha_c, Rn_f, Rn_c.
        """
        if fix not in ("alpha_f", "alpha_c"):
            raise ValueError("fix must be 'alpha_f' or 'alpha_c'.")

        Rn_f = float(self.RNmG_up + self.G)
        Rn_c = float(self.RNmG_down + self.G)

        TaK  = self._K(self.T_a)
        TsfK = self._K(self.T_sf)
        TscK = self._K(self.T_sc)

        sigma = SIGMA_SB
        ea, ef, ec = self.epsilon_a, self.epsilon_f, self.epsilon_c

        LW_down = ea * sigma * TaK ** 4
        LWup_f  = ef * sigma * TsfK ** 4
        LWup_c  = ec * sigma * TscK ** 4

        # Direct algebraic solve (2x2 linear system, no sympy needed):
        #   SW * (1 - alpha_fixed) = Rn_fixed - LW_down + LWup_fixed
        #   alpha_free  = 1 - (Rn_free - LW_down + LWup_free) / SW

        if fix == "alpha_f":
            denom = 1.0 - self.alpha_f
```

```
            if abs(denom) < 1e-15:
                raise RuntimeError("alpha_f ~ 1; cannot solve for SW.")
            SW_val = (Rn_f - LW_down + LWup_f) / denom
            if abs(SW_val) < 1e-15:
                raise RuntimeError("Implied SW ~ 0; cannot solve for alpha_c.")
            alpha_c_val = 1.0 - (Rn_c - LW_down + LWup_c) / SW_val
            alpha_c_val = min(max(alpha_c_val, 0.0), 1.0)

            self.SW_in   = SW_val
            self.alpha_c = alpha_c_val
        else:
            denom = 1.0 - self.alpha_c
            if abs(denom) < 1e-15:
                raise RuntimeError("alpha_c ~ 1; cannot solve for SW.")
            SW_val = (Rn_c - LW_down + LWup_c) / denom
            if abs(SW_val) < 1e-15:
                raise RuntimeError("Implied SW ~ 0; cannot solve for alpha_f.")
            alpha_f_val = 1.0 - (Rn_f - LW_down + LWup_f) / SW_val
            alpha_f_val = min(max(alpha_f_val, 0.0), 1.0)

            self.SW_in   = SW_val
            self.alpha_f = alpha_f_val

        # Dynamic attributes (not dataclass fields -- for diagnostic access)
        self.Rn_c    = Rn_c
        self.Rn_f    = Rn_f
        self.LW_down = LW_down

        return {
            "SW": SW_val,
            "alpha_f": float(self.alpha_f),
            "alpha_c": float(self.alpha_c),
            "Rn_f": Rn_f,
            "Rn_c": Rn_c,
        }

    # ------------------------------------------------------------------
    #  Boundary-condition updates
    # ------------------------------------------------------------------

    def update_surface_BCs_from_reference(
        self,
        *,
        update_T: bool = True,
        update_Q: bool = True,
    ) -> dict:
        """Update wall BCs from fluxes using a two-point neutral log profile.
```

```
        Reference air at ``z_ref = z[0]``, top at ``z_h = Hmax``::

            T_s = T_a + H / (rho cp u* k) * ln(z_h / z_ref)
            Q_s = Q_a + LE / (Lv u* k) * ln(z_h / z_ref)
        """
        z_h      = float(self.Hmax)
        z_ref    = float(self.z[0])
        kappa    = float(self.k)
        log_frac = np.log(z_h / z_ref)

        # cultivated (c)
        T_sc_new = self.T_a + (self.H_c  / (RHO_AIR * CP_AIR * self.ustar_c *␣
↪kappa)) * log_frac
        Q_c_new  = self.Q_a + (self.LE_c / (LV_G * self.ustar_c * kappa)) *␣
↪log_frac

        if update_T:
            self.T_sc = float(T_sc_new)
            self.__dict__.pop("es_c", None)
        if update_Q:
            self.Q_c = float(Q_c_new)

        # fallow (f)
        T_sf_new = self.T_a + (self.H_f  / (RHO_AIR * CP_AIR * self.ustar_f *␣
↪kappa)) * log_frac
        Q_f_new  = self.Q_a + (self.LE_f / (LV_G * self.ustar_f * kappa)) *␣
↪log_frac

        if update_T:
            self.T_sf = float(T_sf_new)
            self.__dict__.pop("es_f", None)
        if update_Q:
            self.Q_f = float(Q_f_new)
            self._apply_Qa_tie()

        return {
            "z_ref": z_ref,
            "T_sc": float(self.T_sc),
            "T_sf": float(self.T_sf),
            "Q_c":  float(self.Q_c),
            "Q_f":  float(self.Q_f),
        }

    def update_surface_BCs_from_zom(
        self,
        *,
```

```
        update_T: bool = True,
        update_Q: bool = True,
    ) -> dict:
        """Like ``update_surface_BCs_from_reference`` but uses z0h = z0m/5."""
        zmax  = float(self.Hmax)
        kappa = float(self.k)

        if self.ustar_c is None or self.ustar_f is None:
            raise ValueError("ustar_f/ustar_c is None; set them or call␣
↪resolve_ustars().")

        z0h_c = self.zom_c / 5
        z0q_c = self.zom_c / 5
        z0h_f = self.zom_f / 5
        z0q_f = self.zom_f / 5

        # cultivated
        T_sc_new = self.T_a + (self.H_c  / (RHO_AIR * CP_AIR * self.ustar_c *␣
↪kappa)) * np.log(zmax / z0h_c)
        Q_c_new  = self.Q_a + (self.LE_c / (LV_G * self.ustar_c * kappa)) * np.
↪log(zmax / z0q_c)

        if update_T:
            self.T_sc = float(T_sc_new)
            self.__dict__.pop("es_c", None)
        if update_Q:
            self.Q_c = float(Q_c_new)

        # fallow
        T_sf_new = self.T_a + (self.H_f  / (RHO_AIR * CP_AIR * self.ustar_f *␣
↪kappa)) * np.log(zmax / z0h_f)
        Q_f_new  = self.Q_a + (self.LE_f / (LV_G * self.ustar_f * kappa)) * np.
↪log(zmax / z0q_f)

        if update_T:
            self.T_sf = float(T_sf_new)
            self.__dict__.pop("es_f", None)
        if update_Q:
            self.Q_f = float(Q_f_new)
            self._apply_Qa_tie()

        return {
            "z_ref": zmax,
            "T_sc": float(self.T_sc),
            "T_sf": float(self.T_sf),
            "Q_c":  float(self.Q_c),
            "Q_f":  float(self.Q_f),
```

```
        }

    # ----------------------------------------------------------------
    #  Heat reconciliation
    # ----------------------------------------------------------------

    def reconcile_heat_with_prescribed_surface_T(
        self,
        *,
        mode: str = "update_H",
        ref: str = "z_ref",
        eps_dT: float = 1e-6,
    ) -> dict:
        """Enforce consistency between (T_sf, T_sc), u*, and fluxes.

        Parameters
        ----------
        mode : {"update_H", "update_ustar"}
            ``"update_H"``    -- keep u* fixed, update H_f and le_factor.
            ``"update_ustar"`` -- keep H_f and le_factor fixed, update u*.
        ref : {"z_ref", "z0h"}
            ``"z_ref"`` uses ln(Hmax / z[0]) (matches uniform_T).
            ``"z0h"``   uses ln(Hmax / (z0m/5)) per surface.
        """
        z_h   = float(self.Hmax)
        kappa = float(self.k)

        def log_fac(surface: str) -> float:
            if ref == "z_ref":
                z_ref = float(self.z[0])
                if z_h <= z_ref:
                    raise ValueError(f"Hmax={z_h} must be > z_ref={z_ref}.")
                return float(np.log(z_h / z_ref))
            elif ref == "z0h":
                z0h = (self.zom_f / 5.0) if surface == "f" else (self.zom_c / 5.
↪0)
                if z_h <= z0h:
                    raise ValueError(f"Hmax={z_h} must be > z0h={z0h} for␣
↪surface={surface}.")
                return float(np.log(z_h / z0h))
            else:
                raise ValueError("ref must be 'z_ref' or 'z0h'.")

        dTf = float(self.T_sf - self.T_a)
        dTc = float(self.T_sc - self.T_a)
        Lf  = log_fac("f")
        Lc  = log_fac("c")
```

```
        if mode == "update_H":
            if self.ustar_f is None:
                raise ValueError("ustar_f is None; set it before␣
↪mode='update_H'.")
            Hf_new = RHO_AIR * CP_AIR * float(self.ustar_f) * kappa * dTf / Lf
            self.H_f = float(Hf_new)

            if self.ustar_c is None:
                raise ValueError("ustar_c is None; set it before␣
↪mode='update_H'.")
            Hc_target = RHO_AIR * CP_AIR * float(self.ustar_c) * kappa * dTc /␣
↪Lc
            RNmG_down = float(self.RNmG_up * self.avail_ratio)
            if abs(RNmG_down) < 1e-12:
                raise ValueError("RNmG_down is ~0; cannot solve for le_factor.")
            self.le_factor = float(1.0 - (Hc_target / RNmG_down))

            return {
                "mode": mode, "ref": ref,
                "H_f_new": float(self.H_f),
                "H_c_target": float(Hc_target),
                "le_factor_new": float(self.le_factor),
                "RNmG_up": float(self.RNmG_up),
                "RNmG_down": float(RNmG_down),
            }

        elif mode == "update_ustar":
            # ustar_f
            if abs(dTf) < eps_dT:
                if abs(self.H_f) > 1e-6:
                    raise ValueError("T_sf ~ T_a but H_f is nonzero.")
                ustar_f_new = 0.0
            else:
                ustar_f_new = float(self.H_f) * Lf / (RHO_AIR * CP_AIR * kappa␣
↪* dTf)
                if ustar_f_new < 0:
                    raise ValueError("Implied ustar_f < 0 (sign mismatch).")
            self.ustar_f = float(ustar_f_new)

            # ustar_c
            Hc_now = float(self.H_c)
            if abs(dTc) < eps_dT:
                if abs(Hc_now) > 1e-6:
                    raise ValueError("T_sc ~ T_a but H_c is nonzero.")
                ustar_c_new = 0.0
            else:
```

```
            ustar_c_new = Hc_now * Lc / (RHO_AIR * CP_AIR * kappa * dTc)
            if ustar_c_new < 0:
                raise ValueError("Implied ustar_c < 0 (sign mismatch).")
        self.ustar_c = float(ustar_c_new)

        return {
            "mode": mode, "ref": ref,
            "ustar_f_new": float(self.ustar_f),
            "ustar_c_new": float(self.ustar_c),
            "H_f": float(self.H_f),
            "H_c": float(Hc_now),
        }
    else:
        raise ValueError("mode must be 'update_H' or 'update_ustar'.")

# ---------------------------------------------------------------------
#  Moisture initialisation & helpers
# ---------------------------------------------------------------------

def __post_init__(self, RH_c: Optional[float], RH_f: Optional[float]):
    # The @property descriptors for RH_c / RH_f shadow the InitVar
    # defaults.  When the caller omits RH_c / RH_f, the auto-generated
    # __init__ passes the property object instead of None.
    if isinstance(RH_c, property):
        RH_c = None
    if isinstance(RH_f, property):
        RH_f = None
    self._init_surface_moisture(RH_c=RH_c, RH_f=RH_f)
    self._apply_Qa_tie()

def _apply_Qa_tie(self):
    """If requested, enforce Q_a = Q_f."""
    if self.tie_Qa_to_Qf:
        self.Q_a = float(self.Q_f)

def _init_surface_moisture(
    self,
    *,
    RH_c: Optional[float],
    RH_f: Optional[float],
):
    self._coerce_surface_pair(label="c", Tc=self.T_sc, RH=RH_c,
                              Q_name="Q_c", RH_default=60.0)
    self._coerce_surface_pair(label="f", Tc=self.T_sf, RH=RH_f,
                              Q_name="Q_f", RH_default=10.0)

def _coerce_surface_pair(
```

```python
        self,
        *,
        label: str,
        Tc: float,
        RH: Optional[float],
        Q_name: str,
        RH_default: float,
        enforce_RH_bounds: bool = True,
    ):
        Q = getattr(self, Q_name)
        if RH is not None and Q is not None:
            raise ValueError(
                f"Specify only ONE of RH_{label} or {Q_name} (not both)."
            )
        if RH is None and Q is None:
            RH = RH_default

        if RH is not None:
            RH = float(RH)
            if enforce_RH_bounds and not (0.0 <= RH <= 100.0):
                raise ValueError(f"RH_{label} must be in [0,100], got {RH}.")
            Q = float(self.Q_from_RH(Tc, RH))
        else:
            Q = float(Q)
            if Q < 0.0:
                raise ValueError(f"{Q_name} must be >= 0, got {Q}.")
            RH_implied = float(self.RH_from_Q(Tc, Q))
            if enforce_RH_bounds and not (0.0 <= RH_implied <= 100.0):
                raise ValueError(
                    f"{Q_name}={Q} at T={Tc} deg C implies "
                    f"RH_{label}={RH_implied:.1f}% (outside [0,100])."
                )
        setattr(self, Q_name, Q)

    @staticmethod
    def Q_from_RH(Tc: float, RH_pct: float) -> float:
        """Q (g m^-3) from T (deg C) and RH (%)."""
        return float(vapor_concentration_RH(Tc, RH_pct))

    @staticmethod
    def RH_from_Q(Tc: float, Q_gm3: float) -> float:
        """RH (%) from T (deg C) and Q (g m^-3)."""
        Tk    = Tc + 273.15
        rho_v = Q_gm3 / 1000.0          # kg m^-3
        e     = rho_v * RV * Tk          # Pa
        es    = saturation_vapor_pressure(Tk)
        return 100.0 * (e / es)
```

```python
    def sync_surface_moisture_inplace(
        self,
        direction: str = "RH_from_Q",
        clip_RH: bool = True,
        clip_Q: bool = True,
    ) -> dict:
        """Return current (T, Q)-consistent RH values for inspection.

        ``direction="RH_from_Q"`` is the only supported mode (RH is a
        read-only property; it is always consistent with Q).
        """
        direction = direction.strip()
        if direction == "RH_from_Q":
            RHc = float(np.clip(self.RH_c, 0.0, 100.0)) if clip_RH else self.
↪RH_c
            RHf = float(np.clip(self.RH_f, 0.0, 100.0)) if clip_RH else self.
↪RH_f
            return {"RH_c": RHc, "RH_f": RHf}
        elif direction == "Q_from_RH":
            raise TypeError(
                "sync_surface_moisture_inplace('Q_from_RH') is no longer␣
↪supported.\n"
                "Use:  p.Q_c = p.Q_from_RH(p.T_sc, new_RH_c)"
            )
        else:
            raise ValueError("direction must be 'RH_from_Q' or 'Q_from_RH'.")

    # ----------------------------------------------------------------
    #  repr
    # ----------------------------------------------------------------

    def __repr__(self) -> str:
        keys = sorted(f.name for f in fields(self))
        body = ",\n  ".join(f"{k}={getattr(self, k)!r}" for k in keys)
        return f"{self.__class__.__name__}(\n  {body}\n)"

    def __str__(self) -> str:
        return self.__repr__()


# ---------------------------------------------------------------------------
# Legacy Params  (backward-compatible with older notebooks)
# ---------------------------------------------------------------------------

@dataclass
class Params:
```

```python
    """Original (pre-AdvectionParams) parameter class.

    Kept for backward compatibility with ``Baldocchi - 1995``,
    ``Advection - grapex``, and other older notebooks.
    New code should use :class:`AdvectionParams`.
    """

    k: float = 0.4
    zom_f: float = 0.005
    zom_c: float = 0.05
    h: float = 2.3
    Lx: float = 2000.0
    Hmax: float = 100.0
    dz: float = 0.5
    dx: float = 5.0
    xmin: float = 0.0

    fallow_fraction: float = 0.5
    fallow_length: float = 1000.0

    T_sc: float = 30.0
    T_sf: float = 50.0
    T_a: float = 18.0
    RH_c: float = 43.0
    RH_f: float = 20.0
    RH_a: float = 18.0

    SW_in: float = 400.0
    e_f: float = 0.95
    e_c: float = 0.95
    e_a: float = 0.80
    alpha_c: float = 0.22
    alpha_f: float = 0.22

    lm_option: str | None = "kz"

    def _grid(self):
        zmin = self.zom_f
        zmax = self.Hmax
        xmax = self.Lx
        x = np.arange(self.xmin, xmax + self.dx, self.dx)
        z = np.arange(zmin + self.dz, zmax + self.dz, self.dz)
        return {
            "x": x, "z": z, "nx": len(x), "nz": len(z),
            "xmax": xmax, "zmin": zmin, "zmax": zmax,
        }
```

```
    def _ustars_from_4m(self):
        Ubar_4 = 3.76
        z_ref = 4.0
        ustar_f = self.k * Ubar_4 / np.log(z_ref / self.zom_f)
        ustar_c = self.k * Ubar_4 / np.log(z_ref / self.zom_c)
        return {"ustar_f": ustar_f, "ustar_c": ustar_c}

    def _block_sizes(self):
        fallow_size = int(self.fallow_length / self.dx)
        field_size = (
            int(fallow_size * (1 - self.fallow_fraction) / self.fallow_fraction)
            if self.fallow_fraction > 0 else 0
        )
        return {"fallow_size": fallow_size, "field_size": field_size}

    def to_dict(self):
        d = asdict(self)
        d.update(self._grid())
        d.update(self._ustars_from_4m())
        d.update(self._block_sizes())
        Tsc_K = self.T_sc + 273.15
        Tsf_K = self.T_sf + 273.15
        es_c = saturation_vapor_pressure(Tsc_K)
        es_f = saturation_vapor_pressure(Tsf_K)
        Q_c = vapor_concentration_RH(self.T_sc, self.RH_c)
        Q_f = vapor_concentration_RH(self.T_sf, self.RH_f)
        Q_a = vapor_concentration_RH(self.T_a, self.RH_a)
        d["Q_c"] = Q_c
        d["Q_f"] = Q_f
        d["Q_a"] = Q_a
        d["es_c"] = es_c
        d["es_f"] = es_f
        return d
```

0.1.8 src/sutton/config_legacy.py

Legacy Params dataclass

[ ]:
```
from dataclasses import dataclass, asdict
import numpy as np
from .physics import saturation_vapor_pressure, vapor_concentration_RH


@dataclass
class Params:
    k: float = 0.4
    zom_f: float = 0.005
```

```
    zom_c: float = 0.05
    h: float = 2.3
    Lx: float = 2000.0
    Hmax: float = 100.0
    dz: float = 0.5
    dx: float = 5.0
    xmin: float = 0.0

    fallow_fraction: float = 0.5
    fallow_length: float = 1000.0

    T_sc: float = 30.0
    T_sf: float = 50.0
    T_a: float = 18.0
    RH_c: float = 43.0
    RH_f: float = 20.0
    RH_a: float = 18.0

    SW_in: float = 400.0
    e_f: float = 0.95
    e_c: float = 0.95
    e_a: float = 0.80
    alpha_c: float = 0.22
    alpha_f: float = 0.22

    lm_option: str | None = "kz"

    def _grid(self):
        zmin = self.zom_f
        zmax = self.Hmax
        xmax = self.Lx
        x = np.arange(self.xmin, xmax + self.dx, self.dx)
        z = np.arange(zmin + self.dz, zmax + self.dz, self.dz)
        return {"x": x, "z": z, "nx": len(x), "nz": len(z), "xmax": xmax,␣
↪"zmin": zmin, "zmax": zmax}

    def _ustars_from_4m(self):
        Ubar_4 = 3.76
        z_ref = 4.0
        ustar_f = self.k * Ubar_4 / np.log(z_ref / self.zom_f)
        ustar_c = self.k * Ubar_4 / np.log(z_ref / self.zom_c)
        return {"ustar_f": ustar_f, "ustar_c": ustar_c}

    def _block_sizes(self):
        fallow_size = int(self.fallow_length / self.dx)
        field_size = int(fallow_size * (1 - self.fallow_fraction) / self.
↪fallow_fraction) if self.fallow_fraction > 0 else 0
```

```
        return {"fallow_size": fallow_size, "field_size": field_size}

    def to_dict(self):
        d = asdict(self)
        d.update(self._grid())
        d.update(self._ustars_from_4m())
        d.update(self._block_sizes())
        Tsc_K = self.T_sc + 273.15
        Tsf_K = self.T_sf + 273.15
        es_c = saturation_vapor_pressure(Tsc_K)
        es_f = saturation_vapor_pressure(Tsf_K)
        Q_c = vapor_concentration_RH(self.T_sc, self.RH_c)
        Q_f = vapor_concentration_RH(self.T_sf, self.RH_f)
        Q_a = vapor_concentration_RH(self.T_a, self.RH_a)
        d["Q_c"] = Q_c
        d["Q_f"] = Q_f
        d["Q_a"] = Q_a
        d["es_c"] = es_c
        d["es_f"] = es_f
        return d
```

0.1.9 src/sutton/__init__.py

Package ***init*** *(public API)*

[ ]:
```
from .config import Params, AdvectionParams
from .constants import SIGMA_SB, CP_AIR, RHO_AIR, LV, LV_G, RV
from .numerics import thomas, thomas_alt, our_central_difference,␣
 ↪no_central_difference
from .physics import saturation_vapor_pressure, vapor_concentration_RH,␣
 ↪vapor_concentration, e_sat
from .integrators import integrate_T_implicit, integrate_H2O_implicit
from .stability import stability
from .utils import padit

__all__ = [
    "Params", "AdvectionParams",
    "SIGMA_SB", "CP_AIR", "RHO_AIR", "LV", "LV_G", "RV",
    "thomas", "thomas_alt", "our_central_difference",
    "no_central_difference",
    "saturation_vapor_pressure", "vapor_concentration_RH",␣
 ↪"vapor_concentration", "e_sat",
    "integrate_T_implicit", "integrate_H2O_implicit",
    "stability",
    "padit",
]
```